

코딩 에이전트란 무엇이고 퀀트 작업에서 무엇을 할 수 있는가

코딩 에이전트의 작업 방식부터 사람이 직접 확인할 일까지

확인일 2026-08-27

코딩 에이전트는 파일을 살피고 도구를 써서 결과를 만든다

확인하고 실행하고 다시 확인하는 일을 되풀이한다

이 리포트에서 **코딩 에이전트(coding agent)**는 사용자의 요청을 바탕으로 관련 파일을 찾고, 필요한 도구를 골라 실행하며, 그 결과를 보고 다음 작업을 정하는 AI를 뜻한다. 일반 대화형 AI가 설명이나 예시 코드를 건네는 데 그칠 수 있다면, 코딩 에이전트는 허용된 작업 폴더 안에서 실제 파일을 만들거나 고치고 명령을 실행할 수 있다.

OpenAI의 Codex IDE 공식 문서에 따르면 에이전트는 편집기에 열린 파일과 선택한 코드를 참고해 파일을 설명하거나 고칠 수 있다. 바뀐 내용은 같은 편집기에서 확인할 수 있고, 시간이 더 걸리는 작업은 별도 환경에 맡긴 뒤 결과만 가져와 검토할 수도 있다 (Codex IDE 확장 기능 공식 문서).

이런 작업 순환을 이해하는 데 **ReAct(Reason + Act)**라는 연구 틀이 도움이 된다. ReAct는 언어 모델이 다음에 할 일을 판단해 외부 환경에서 실행하고, 실행 결과를 관찰한 뒤 다음 판단에 반영하는 과정을 설명한다 (ReAct 논문). 여기서는 ReAct를 Codex의 내부 구조라고 단정하지 않는다. 파일과 명령을 사용하는 에이전트가 어떻게 여러 단계를 이어 가는지 이해하기 위한 일반적인 설명 틀로만 사용한다.

그림 1

코딩 에이전트의 기본 작업 순환. ReAct의 판단·행동·관찰 구조를 파일과 명령을 사용하는 작업에 맞게 풀어 그렸다. Codex의 실제 내부 설계도가 아니라 학습을 위한 개념도다.



그림에서 **현재 상태**는 사용자의 요청, 열린 파일, 기존 코드, 앞서 실행한 명령의 결과처럼 에이전트가 지금 참고할 수 있는 정보다. 에이전트는 이 정보를 보고 다음 작업을 정한 뒤 파일 도구나 명령 도구를 사용한다. 실행 결과나 오류가 돌아오면 다시 상태를 살피고, 일이 끝날 때까지 이 과정을 반복한다. 마지막에는 사람이 바뀐 파일과 실행 결과를 확인한다.

에이전트라는 이름이 “혼자서 언제나 완벽하게 끝낸다”는 뜻은 아니다. 잘못된 계획을 세우거나 관련 파일을 놓칠 수 있고, 명령이 실패할 수도 있다. 코딩 에이전트의 장점은 사람의 확인을 없애는 데 있지 않다. **확인할 수 있는 파일과 실행 기록을 남기면서 반복 작업을 줄여 주는 데 있다.**

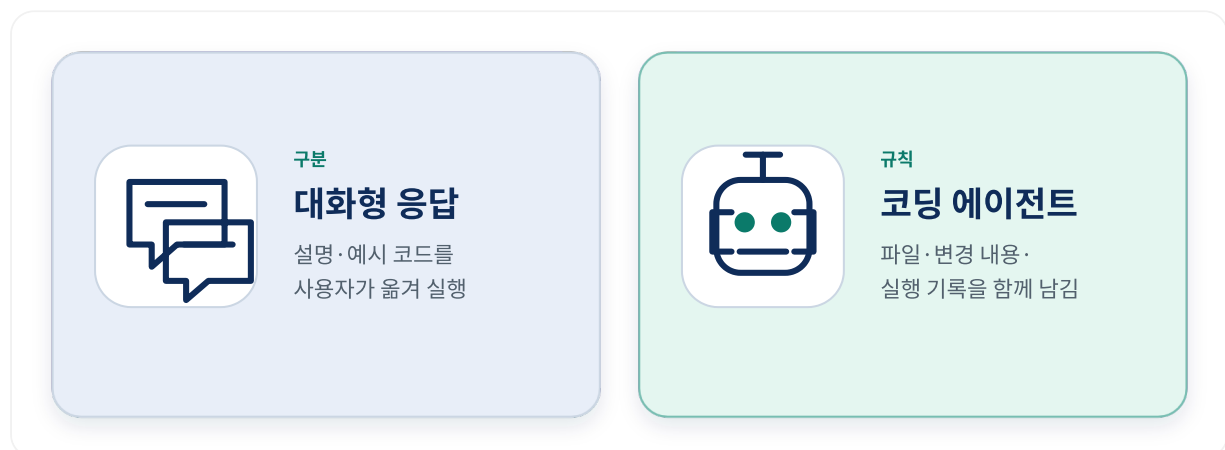
대화형 AI와 코딩 에이전트는 남기는 결과가 다르다

같은 AI라도 연결된 도구에 따라 할 수 있는 일이 달라진다

“챗봇”과 “코딩 에이전트”를 서로 완전히 다른 제품으로 외우면 오히려 헷갈린다. 같은 AI 서비스도 대화창에서는 설명을 주고받는 데 집중할 수 있고, 코드 편집기와 작업 폴더에 연결되면 파일을 고치고 명령을 실행하는 에이전트처럼 일할 수 있다. 중요한 차이는 제품 이름이 아니라 **무엇을 보고, 무엇을 바꾸며, 무엇을 결과로 남기는가다.**

그림 2

대화형 응답과 에이전트 작업에서 남는 결과의 차이. Codex IDE와 prompting 공식 문서의 기능 구분을 초보자용 두 흐름으로 재구성.



대화형 응답만 받을 때는 사용자가 코드를 복사하고, 파일 위치를 고르고, 실행하고, 오류를 다시 전달해야 한다. 에이전트 작업에서는 AI가 허용된 폴더를 살펴보고 파일을 고친 다음, 명령을 실행해 오류를 읽고 수정할 수 있다. OpenAI는 Codex를 작은 요청에 실시간으로 협업하는 방식과 더 긴 작업을 맡아 수행하는 방식에 모두 쓰는 코딩 에이전트로 설명한다 (Introducing upgrades to Codex).

표 1 제품 이름이 아니라 참고 정보·행동·남는 결과로 구분한 두 사용 방식.

구분	대화형 응답에 머무를 때	코딩 에이전트로 작업할 때
주로 보는 것	사용자가 붙여 넣은 질문과 자료	질문, 열린 파일, 선택 영역, 작업 폴더의 관련 파일
주로 하는 일	설명·예시 코드 제안	파일 탐색·수정, 명령 실행, 오류 확인, 재수정
남는 결과	대화 내용	대화 내용, 실제 파일, 바뀐 내용, 실행 기록
사람이 할 일	코드를 옮기고 실행하는 일까지 직접 수행	작업 목표를 정하고 변경과 실행 결과를 검토

이 표는 제품을 딱 잘라 나누려는 분류표가 아니다. 대화형 AI도 연결된 도구에 따라 파일을 다룰 수 있고, 코딩 에이전트도 모든 요청에서 명령을 실행하는 것은 아니다. 한 작업 안에서도 먼저 설명을 듣고 계획을 고친 뒤 파일 변경을 허용할 수 있다. 사람은 작업의 위험과 준비 상태에 따라 어느 단계까지 맡길지 정한다.

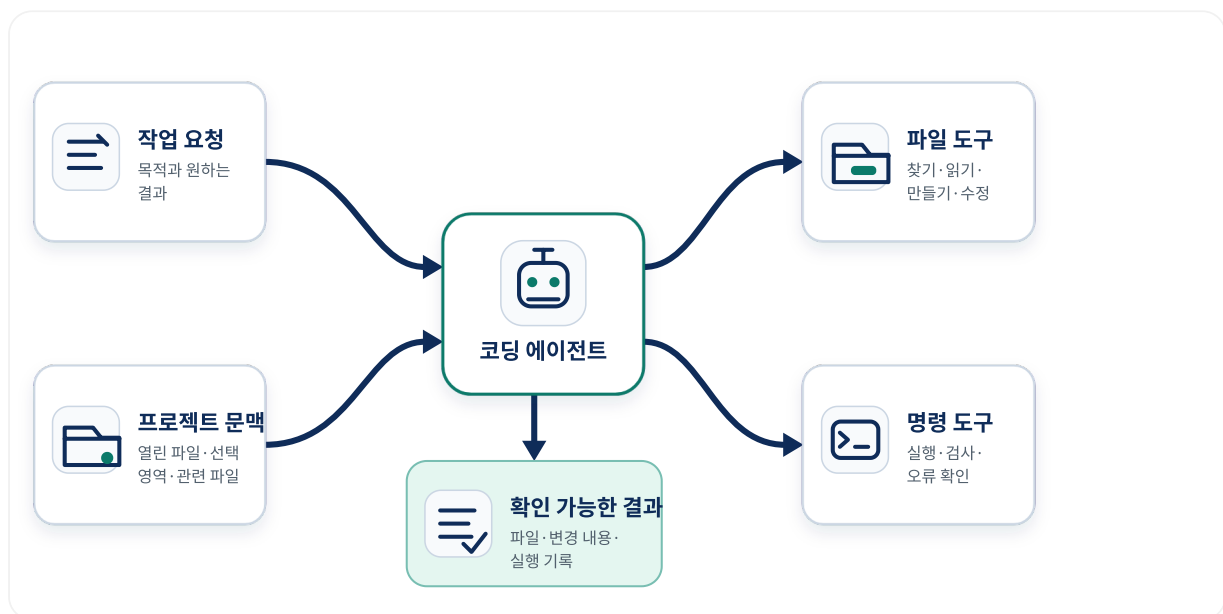
에이전트는 작업 폴더를 살피고 필요한 도구를 사용한다

작업 문맥은 에이전트가 이번 일에 참고할 정보다

문맥(context)은 에이전트가 이번 작업을 이해할 때 참고하는 정보다. 사용자의 요청, 현재 열어 둔 파일, 선택한 코드, 폴더 안의 안내 문서와 기존 파일이 여기에 들어갈 수 있다. Codex IDE 확장 기능에서는 열린 파일과 선택 영역을 대화에 넣고, 바뀐 내용을 같은 화면에서 확인할 수 있다 (Codex IDE 확장 기능 공식 문서).

그림 3

코딩 에이전트가 일할 때 참고하는 정보와 사용하는 도구. Codex IDE 공식 문서의 기능을 초보자가 이해하기 쉬운 관계도로 재구성.



문맥은 많지만 하면 좋은 것이 아니다. 다른 종목의 오래된 노트북, 관계없는 데이터 파일, 서로 충돌하는 지시가 함께 들어가면 엉뚱한 기준을 따를 수 있다. 초보자는 “내 컴퓨터 전체를 AI가 다 안다”고 생각하기 쉽지만, 실제로는 **현재 열어 준 프로젝트와 전달한 정보 안에서 판단한다고** 이해해야 한다.

에이전트는 관련 파일을 찾고 계획을 세운 뒤 파일을 고치고 명령을 실행한다. 실패하면 오류를 읽고 다시 수정한다. OpenAI의 prompting 공식 문서도 버그를 고칠 때 재현 방법과 제약을 알려 주고, 수정한 뒤에는 같은 절차와 관련 검사를 다시 실행하라고

권한다 (Prompting Codex).

여기서 **명령(command)**은 컴퓨터에 시키는 구체적인 실행 지시다. 예를 들어 Python 파일을 실행하거나 테스트를 돌리고 폴더 안의 파일 목록을 확인하는 일이 명령이다. 명령이 성공했다는 사실은 해당 명령이 오류 없이 끝났다는 뜻일 뿐, 투자 목적이 옳거나 데이터가 정확하다는 뜻은 아니다.

표 2 에이전트의 행동을 사람이 확인할 수 있는 흔적과 연결한 표.

에이전트의 행동	화면이나 파일에 남아야 할 기록	사람이 확인할 질문
관련 파일을 찾는다	읽은 파일 이름과 작업 범위	관계없는 폴더까지 보지 않았는가
파일을 만든다· 고친다	새 파일과 변경 전후 차이	요청한 이름과 내용이 맞는가
명령을 실행한다	실행한 명령과 종료 결과	필요한 검사를 실제로 실행했는가
오류를 수정한다	실패 원인과 고친 내용	겉으로 드러난 오류만 덮지 않고 원인을 고쳤는가
완료를 보고한다	만든 파일·검사 결과·남은 문제 요약	보고와 실제 결과가 일치하는가

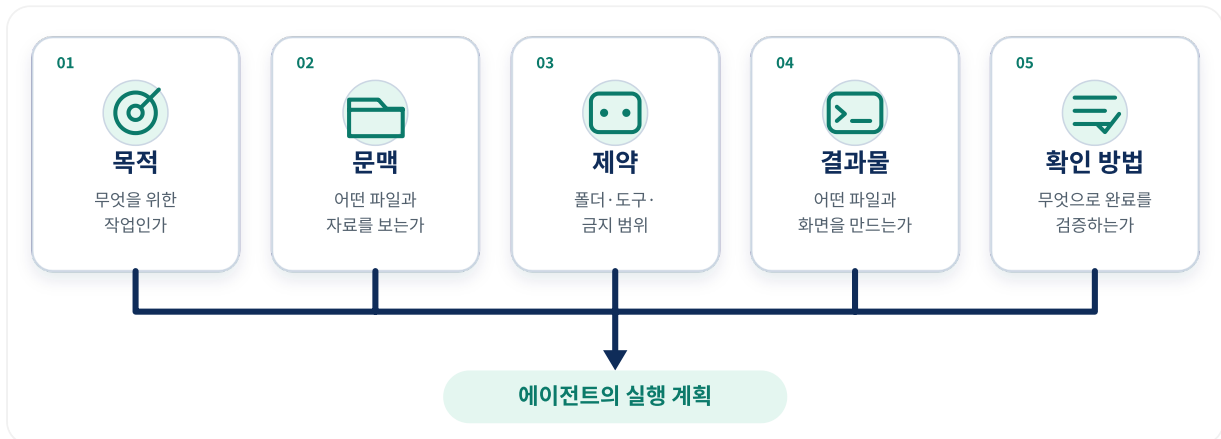
좋은 작업 요청은 목적과 범위와 확인 방법을 함께 말한다

Python 문법 대신 원하는 결과를 구체적으로 말한다

코딩을 모른다고 해서 요청을 막연하게 해도 되는 것은 아니다. Python 문법은 몰라도 **무엇을, 어디에서, 어떤 파일로 만들고, 어떻게 확인할지**는 투자자가 정할 수 있다.

OpenAI의 Codex prompting 문서는 유용한 요청에 원하는 동작, 관련 코드나 재현 절차, 지켜야 할 제약, 변경을 확인하는 방법이 들어간다고 설명한다 (Prompting Codex).

그림 4 초보자가 작업 요청에 넣을 다섯 요소. OpenAI의 프롬프트 작성 지침을 퀴트 실습용 질문으로 재구성.



다음 예시는 영상 4에서 사용할 KODEX 200 작업을 **설계만** 해 보는 것이다. 아직 패키지를 설치하거나 가격을 내려받지 않는다.

표 3 짧지만 모호한 요청을 확인 가능한 작업 요청으로 바꾸는 세 가지 예.

모호한 요청	무엇이 빠졌는가	고쳐서 말할 내용
“KODEX 200 차트 만들어 줘”	종목코드·기간·저장 위치·완료 조건이 없다	069500, 날짜 범위, CSV와 노트북, 표와 증가 그래프를 지정한다
“알아서 좋은 방식으로 해 줘”	에이전트가 데이터 경로와 파일 구조를 스스로 정하게 된다	현재 작업 폴더만 쓰고 계획을 먼저 설명하라고 한다
“됐으면 알려 줘”	무엇을 확인해야 완료인지 알 수 없다	만든 파일, 실행 명령, 결과, 남은 한계를 보고하게 한다

한 번에 완벽한 요청을 쓸 필요는 없다. 먼저 계획만 부탁하고 빠진 항목을 확인한 다음 파일 변경을 허용해도 된다. 중요한 것은 에이전트가 대신 정한 조건을 내가 정한 조건으로 착각하지 않는 것이다.

읽기 포인트

영상 4에서는 이렇게 요청한다: 현재 강의 폴더 안에서만 작업한다. KODEX 200 069500 의 정해진 기간 가격을 로컬 CSV에 저장하고, 노트북이 그 CSV를 다시 읽어 표와 종가 그래프를 표시하게 한다. 먼저 계획과 필요한 파일을 설명하고, 끝나면 만든 파일·실행 방법·확인 결과·남은 한계를 보고한다.

이 문장은 수익 전략을 만들어 달라는 요청이 아니다. 데이터 파일과 표시 노트북을 만드는 작업만 정의한다. 어떤 때 사고팔지, 결과가 수익성이 있는지 판단하는 일은 들어 있지 않다.

사람은 방향을 정하고 에이전트는 정해진 일을 수행한다

일을 나누면 누가 무엇을 책임질지 분명해진다

코딩 에이전트를 잘 쓴다고 해서 모든 일을 맡길 필요는 없다. 사람은 투자 대상과 목적, 허용 범위, 완료 조건을 정한다. 에이전트는 그 범위 안에서 파일을 살피고 코드를 만들고 실행한다. 마지막에는 사람이 바뀐 내용과 실행 결과를 보고 받아들일지, 더 고치게 할지, 되돌릴지 정한다.

그림 5

사람과 코딩 에이전트가 일을 나누는 흐름. OpenAI의 검토 권고와 AIML Quant의 초보자용 역할 구분을 결합해 재구성.



OpenAI도 에이전트가 바뀐 내용을 적용하거나 배포하기 전에 사람이 검토하라고 권한다. Codex가 남긴 작업 기록과 테스트 결과는 이때 참고할 근거가 된다. 코드 리뷰 기능 역시 사람의 검토를 대신하는 기능이 아니라 검토를 돕는 보조 수단으로 쓰라고 밝힌다 (Introducing upgrades to Codex).

표 4

사람과 에이전트가 맡을 일을 목표·범위·구현·검증·최종 결정으로 나눈 표. 이 역할 구분은 AIML Quant의 학습용 기준이다.

판단과 작업	사람이 책임진다	에이전트가 돕는다
목표	어떤 종목·기간·결과물이 필요한지 정한다	목표를 실행할 단계로 나누어 제안한다
범위	어느 폴더와 어떤 명령까지 허용할지 정한다	허용된 범위 안에서 파일과 도구를 사용한다
구현	중요한 금융 가정과 완료 조건을 확인한다	코드와 파일의 초안을 만들고 오류를 고친다
검증	결과가 요청·데이터·투자 목적과 맞는지 판단한다	실행 기록과 검사 결과를 모아 보고한다
최종 결정	수락·수정·되돌리기·중단을 결정한다	후속 요청에 따라 다시 작업한다

퀀트 작업에서는 이 구분이 특히 중요하다. 에이전트가 종가 그래프를 정확히 그렸더라도 그 그래프만으로 매수할지 결정해서는 안 된다. 데이터에 미래 정보가 섞였는지, 전략 규칙이 투자 아이디어를 제대로 나타내는지, 손실 위험을 감수할 수 있는지는 사람이 따로 판단해야 한다.

완료 보고만 믿지 말고 파일과 실행 결과를 확인한다

“완료했습니다”라는 문장을 그대로 믿지 않는다

에이전트의 마지막 답변은 작업 결과를 찾기 쉽게 요약한 **보고다**. 그 문장만으로는 작업이 끝났다고 볼 수 없다. 파일이 실제로 생겼는지, 바뀐 내용이 요청과 맞는지, 명령을 실행했는지, 표와 그래프가 나타나는지 차례로 확인해야 한다.

그림 6

완료 여부를 확인하는 다섯 단계. Codex 공식 문서의 변경 검토·명령 기록·테스트 결과 권고를 초보자용 순서로 재구성.



OpenAI의 IDE 문서는 변경 요약과 실제로 바뀐 줄을 함께 확인하고, 원하는 변경만 남긴 뒤, 필요하면 같은 대화에서 다시 요청하라고 안내한다. 첫 작업 전후에 Git 체크포인트를 만들어 되돌릴 수 있게 하라는 권고도 포함한다 (Codex IDE 확장 기능 공식 문서). Git은 파일의 변경 이력을 기록하는 도구다. 영상 3에서는 설치와 실행 환경에 집중하고, Git의 자세한 사용법은 이후 보충 자료에서 다룬다.

확인 순서는 다음 순서면 충분하다.

1. **파일** — 요청한 이름과 위치에 실제 파일이 있는가.
2. **변경 내용** — 새로 들어간 내용이 요청한 범위를 벗어나지 않았는가.
3. **실행 기록** — 에이전트가 말한 명령을 실제로 실행했는가.

4. **화면에 나온 결과** — 표, 그래프, 오류 메시지를 내가 직접 볼 수 있는가.

5. **요구 조건** — 종목, 기간, 파일 형식, 표시 내용이 처음 요청과 같은가.

테스트가 통과해도 모든 요구가 맞는 것은 아니고, 화면이 그럴듯해도 데이터가 맞는 것은 아니다. 각 증거는 서로 다른 질문에 답한다. 영상 4에서는 노트북을 처음부터 다시 실행해 앞서 실행한 셀이나 임시 상태가 없어도 같은 결과가 나오는지 확인한다.

에이전트에게는 필요한 권한만 준다

에이전트가 할 수 있는 일과 해도 되는 일은 다르다

코딩 에이전트는 명령을 실행할 수 있으므로 편리하지만, 그만큼 허용 범위를 먼저 정해야 한다. Codex의 공식 prompting 문서는 로컬 명령이 파일과 네트워크 접근을 제한하는 샌드박스 안에서 실행되고, 제한 범위를 넘어야 할 때는 사용자의 승인 정책을 따른다고 설명한다 (Prompting Codex). OpenAI의 실제 운영 사례도 위험이 낮은 일상 작업은 정해진 범위 안에서 처리하고, 위험이 커지는 행동은 사람의 검토에서 멈추게 하는 원칙을 설명한다 (Running Codex safely at OpenAI).

그림 7

초보자 실습에서 어디까지 맡길지 정하는 기준. 강의 폴더 안의 필요한 작업과 추가 확인이 필요한 작업을 나눈다. 제품의 세부 설정값이 아니라 이번 강의의 안전 원칙을 보여 준다.



이 과정의 첫 실습에서는 강의 전용 폴더 안에서만 파일을 만들고 고친다. 비밀번호, API 키, 증권사 인증 정보는 프롬프트나 노트북에 넣지 않는다. 에이전트에게 실제 주문 전송을 맡기지 않고, 삭제처럼 되돌리기 어려운 작업도 하지 않는다. 인터넷에서 데이터를 받아야 하는 영상 4에서는 어떤 사이트와 패키지를 쓸지 계획부터 확인한 다음 사용을 허용한다.

읽기 포인트

안전 원칙: 처음부터 넓은 권한을 줄 필요는 없다. 작업 폴더, 필요한 파일, 필요한 명령을 좁혀 주면 결과를 확인하고 되돌리기도 쉬워진다.

용어와 근거

핵심 용어

용어	이 리포트에서의 뜻
AI 모델	입력을 바탕으로 설명이나 코드 같은 출력을 만드는 기본 모델
코딩 에이전트	요청과 현재 상태를 바탕으로 도구를 사용하고 그 결과를 확인하며 여러 단계의 작업을 이어 가는 AI
작업 문맥	현재 요청을 이해하기 위해 에이전트가 참고하는 열린 파일, 선택 영역, 프로젝트 파일, 안내 문서
작업 폴더	이번 과제의 파일을 모아 두고 에이전트의 작업 범위로 지정한 폴더
명령	파일 실행, 패키지 확인, 테스트처럼 컴퓨터에 구체적인 행동을 시키는 지시
변경 내역	어떤 파일의 어느 부분이 전과 달라졌는지 보여 주는 기록
샌드박스	에이전트가 파일·명령·네트워크를 사용할 수 있는 범위를 기술적으로 제한한 환경
승인	정해진 범위를 넘는 작업을 계속할지 사람이 확인하고 허용하는 절차
ReAct	다음에 할 일을 판단하고 실행한 뒤 그 결과를 확인하는 과정을 되풀이하는 에이전트 연구들. 이 리포트에서는 일반적인 작업 순환을 설명하는 데만 사용

이 리포트의 정의와 한계

요청과 현재 상태를 살피고, 도구를 사용하고, 결과를 다시 확인하는 흐름은 여러 제품에 공통으로 적용되는 표준 정의가 아니다. Codex의 공식 기능과 ReAct 연구를 초보자가 이해하기 쉽도록 AIML Quant가 정리한 **학습용 구조**다. 특히 ReAct는 Codex의 내부 구현을 설명하는 자료가 아니다. 제품의 기능과 화면은 바뀔 수 있으므로 영상 3을 촬영할 때 설치 방법과 권한 화면을 다시 확인한다.

OpenAI가 공개한 내부 개발 사례는 사람이 목표와 검증 기준을 정하고 에이전트가 구현을 수행하는 작업 방식을 보여 준다. 다만 그것은 특정 조직과 개발 환경의 사례이지, 모든 초보자나 모든 투자 작업에서 같은 생산성이나 품질이 나온다는 증거는 아니다 (Harness engineering).

외부 1차 자료에서 확인한 내용

자료	이 리포트에서 확인한 내용
OpenAI, Codex IDE 확장 기능 공식 문서	열린 파일·선택 영역을 참고하고, 설명·수정·변경 검토·긴 작업 위임을 편집기 안에서 이어서 할 수 있다는 점
OpenAI, Prompting Codex	작업 요청에 원하는 동작, 관련 문맥, 재현 절차, 제약, 검증 방법을 넣고 수정 뒤 관련 검사를 다시 실행하라는 지침
OpenAI, Custom instructions with AGENTS.md	에이전트가 작업 전에 프로젝트별 안내 파일을 읽어 반복되는 규칙과 문맥을 적용할 수 있다는 점
OpenAI, Introducing upgrades to Codex	Codex가 파일 수정·테스트·코드 리뷰를 수행할 수 있지만 적용과 배포 전에는 사람이 결과를 검토해야 한다는 권고
OpenAI, Running Codex safely at OpenAI	허용 범위, 사람의 승인, 기록을 함께 사용해 위험이 낮은 행동과 추가 검토가 필요한 행동을 나눈 운영 원칙
OpenAI, Harness engineering	사람이 목표·완료 조건·검증을 맡고 에이전트가 구현을 맡는 역할 분담의 실제 조직 사례와 그 사례의 범위
Yao 외, ReAct: Synergizing Reasoning and Acting in Language Models	언어 모델이 판단과 행동을 번갈아 수행하고, 행동으로 얻은 관찰을 다음 판단에 반영하는 일반적인 에이전트 연구 틀

표 1~4와 그림 1~7의 구분과 점검 순서는 공식 문서의 문장을 그대로 옮긴 제품 사양이 아니라, 코딩 경험이 없는 투자자가 작업에서 빠진 부분을 찾도록 AIML Quant가 만든 학습용 설명이다. 이 기준을 따른다고 코드 정확성, 데이터 정확성, 투자 수익성 또는 안전한 실전 운용이 보장되지는 않는다.

강의 자료 내려받기

상세 리포트와 발표자료의 PDF, 목적·범위·결과물·확인 방법을 빠짐없이 적도록 만든 요청문 양식을 제공한다. 대괄호 안의 내용을 자신의 작업에 맞게 바꿔 사용한다.

- 상세 리포트 PDF 내려받기
- 발표자료 PDF 내려받기
- 코딩 에이전트 작업 요청문 양식 내려받기

요청문을 보낸 뒤에는 에이전트의 완료 요약만 읽고 끝내지 않는다. 실제 파일과 변경 내용, 실행 결과가 처음 요청한 조건과 맞는지 직접 확인한다.